Introduction

Before jumping in, let's talk about what we're actually learning in this course.

WE'LL COVER THE FOLLOWING

- Preface
- GitHub
- How we'll do this

Preface

CSS isn't what it used to be. Today, you'll be hard-pressed to find a strong, straightforward, and expressive UI language that can beat modern CSS.

As the tech world moves more and more towards web-based frameworks for UI, seen in the embrace of electronics for modern sleek looking desktop apps, we start to see a bigger emphasis on highly customized looks for apps. Everyone's racing to create that *dark theme*, right?

The goal of this course is two-fold:

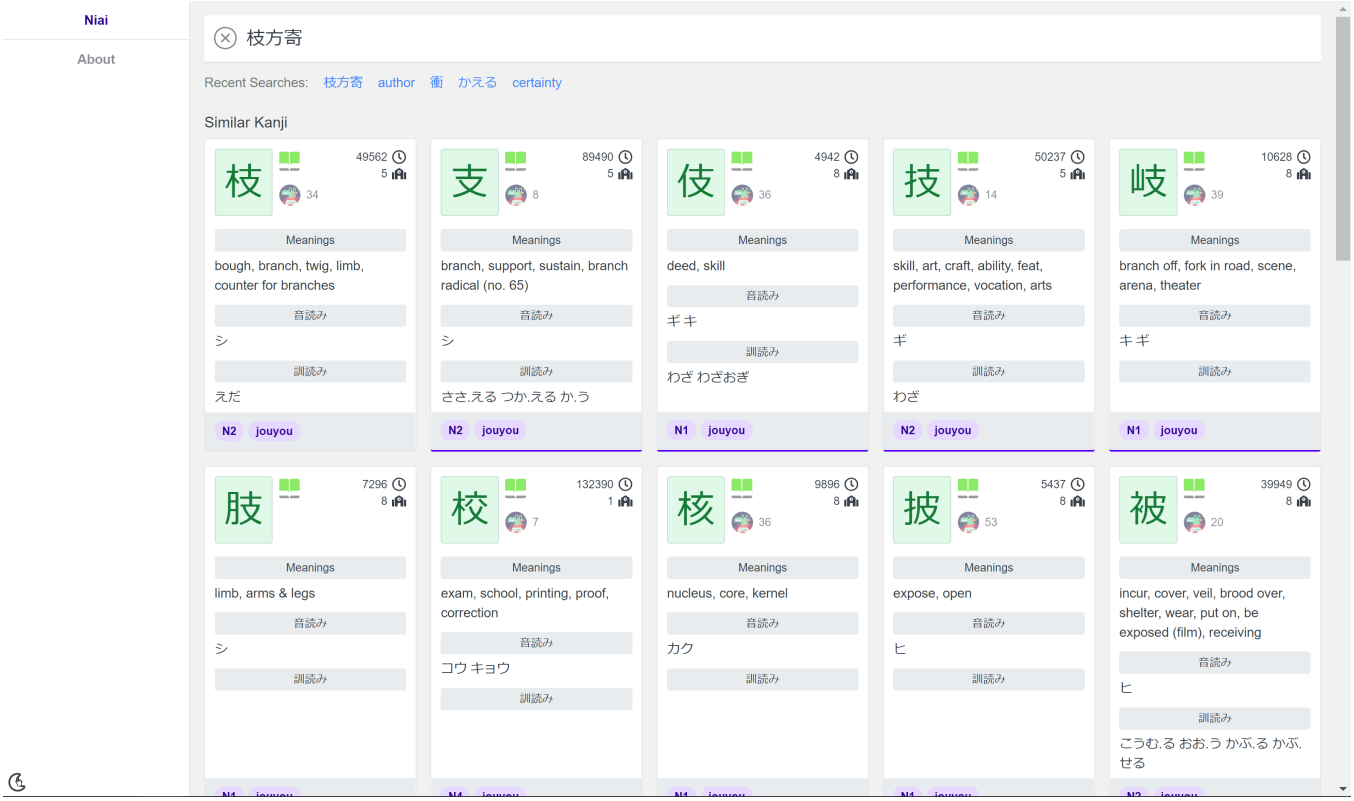
- Devise practical methods and patterns around theming and outline some common problems and their solutions.
- Learn how to use `css-theming`. We wouldn't want to repeat the theming plumbing each time we build a new app, so `css-theming` is a publicly distributed package that implements all patterns discussed.

While discussing problems and solutions in the first part, we'll be building our *theoretical* system, the system that is implemented in `css-theming`.

As such, this course is not a documentation of CSS features! It's rather the practical results of my experience over years developing highly customized complex UIs using CSS. I will express some opinionated views, but you can use

the knowledge obtained to develop any variation if you so desire.

Some of our discussions will be around colors, as this is the most important part of a theme. Something I want to emphasize here is that developing a consistent system that can also understand different **brightness** levels is hard and requires intricate planning. For example, Slack has had support for themes since the start but only introduced a true *dark* theme not long ago. There's an inherit complexity in designing a well thought out *dark* theme as we'll see.



from niai.mrahhal.net

But a *dark* theme is all the rage today and it's being embraced more and more as time goes on, so our system will definitely have to support this. In addition, the system we create must make it easy to reason about what color/shade/swatch to use when and where.

Finally, we're going to use SCSS on top of CSS to make our life a bit easier. SCSS is a superset of CSS that gives us more control than CSS. It won't be an essential piece here, and you're welcome to use LESS or any other framework, but `css-theming` is implemented using SCSS.

GitHub

This course comes with the css-theming github repo. You'll use the `css-theming` package to access the SCSS and JS/TS tools in your project.

`theming` package to access the SCSS and JS/TS tools in your project.

How we'll do this

I like to hammer out the theory before implementation. In the first part, we'll learn multiple different patterns and solve problems around theming. The second part is Using css-theming; this will act as a full guide to using and customizing `css-theming` to suit every app's needs.

Now that you're familiar with what we'll be learning throughout this course, let's dig deep into the main concepts for mastering CSS theming.

Introduction to CSS Theming

This lesson provides a formal introduction to CSS theming and discusses what concepts are to be covered later in this chapter.

WE'LL COVER THE FOLLOWING

- Introduction
- What is CSS theming
- Theme props
- Notions as CSS variables

Introduction

In this chapter, we'll talk about several common problems around theming and how to solve them. We'll introduce some opinionated notions that'll be the basis behind creating a well-defined theming platform.

What is CSS theming

We all know what a *theme* is; they are kind of like different *clothes* we can dress our app in. The app's function doesn't change, but its look and colors do. In CSS theming, we use facilities in CSS to accomplish this.

Theme props

A *theme* has properties that we call *theme props*. In practice, these props are CSS variables.

> **Note:** We'll talk more about CSS variables and how we'll use them in the next lesson.

As consumers of such a theming platform, all we need to do to use a theme

prop is simply use the CSS variable it represents.

For a simple example, let's imagine that our theming platform defines a `--text-color` prop, this supposedly defines a color suitable for use as the app's main text color. In this case, using it involves setting our body's text color to this variable:

```
body {
  color: var(--text-color);
}
```

This variable will evaluate to whatever value that makes sense for the currently applied theme; for example, blackish color for light themes, whitish color for dark themes.

We shouldn't have to worry about what the current theme is. We simply know that this variable will be appropriate.

As we'll see later, things get more complicated when we start talking about a palette of colors with shades/swatches, which is why having a platform that solves these small problems that intertwine is important.

Notions as CSS variables

In each lesson, we will introduce a certain notion and link it to a CSS variable name which, as we explained, also represents a theme prop. This will make it easier to talk about the concepts.

For example, the main background color is referred to by using the following CSS variable: `--bg-main`. This means, there will be a `--bg-main` prop on our theme that refers to the main background of our app.

You can use the information in this section to create a custom solution if you want. But as we'll see later, the `css-theming` package will be more than enough for the majority of apps. Understanding the concepts and theories in this section will allow you to more easily customize `css-theming` to your liking. This package can be considered the practical implementation of this section.

CSS Variables

This lesson introduces CSS variables and how we'll use them in our themes.

WE'LL COVER THE FOLLOWING

- Inheritance
- Dynamically creating variables

> **Note:** If you need a quick refresher about CSS variables, check this shot.

Inheritance

CSS variables (also called custom properties) have one special feature that makes them perfect for our theming needs. CSS variables are *inherited*.

Consider the following example:

HTML CSS Output

```html
<html>
 <head>
 </head>
 <body>
   <div class="outer">
     <div class="foo">

     <div class="inner
       <div class="foo
     </div>
   </div>
 </body>
</html>
```

```css
.outer,
.inner {
    margin: 5px;
    border: 1px solid #333;
    border-radius: 3px;
}

.outer {
    --some-color: red;
}

.inner {
    --some-color: blue;
}

.foo {
    color: var(--some-color);
}
```

Foo

Foo

When `var(--some-color)` is being evaluated, the hierarchical HTML tree is traversed until a definition of `--some-color` is found. This is great, as it means we can scope CSS variables to a certain section of our app, and also override them in an inner component if we need to.

This is all that we need for now when it comes to CSS variables. We'll be using them more when we define our themes later. What we call a **theme prop** is actually a CSS variable on the theme definition, and we'll be using those terms interchangeably.

Dynamically creating variables

Lastly, let's talk a bit about how we'll create those variables.

Most of our props will be computed on the go according to theme values. In SCSS, creating a CSS variable that we compute from some other SCSS value is super easy:

```
// Assume we get this from the theme
$color: red;

body {
    --color: #{$color};
}
```

With the power of SCSS, we'll be able to do all kinds of computations, and then create CSS variables with the results for easy consumption. Once you've created a CSS variable, the next step is to learn how to apply them conditionally. Move to the next lesson to learn this.

Defining and Using Theme Props

In this lesson we'll discuss how to define our themes and how to use a theme's props in a simple example.

WE'LL COVER THE FOLLOWING

- Example: Toggling between dark and light mode

Having CSS variables change values between different themes is the main way by which our theming platform will work. This lesson includes a simple example that toggles between a light and a dark theme of an app. This example will also show what we mean by *defining a theme* and how we'll use the theme props in our app.

Example: Toggling between dark and light mode

Let's assume our themes have 2 props, `--background` (background color) and `--color` (text color), and that we have 2 themes we want to define, *light* and *dark*.

It's easy to switch themes by assigning a unique CSS class for each theme and attaching the theme we want to activate on the HTML element. This, in turn, applies the CSS variables to the target container of the theme, a notion we'll talk more about later.

The most basic example of this is:

Output

JavaScript

HTML

CSS (SCSS)

Toggle Theme

Hello!

Let's start with the SCSS. We first define our theme variables:

```
// Definition of our light theme:
.theme-light body {
    --background: white;
    --color: black;
}

// Definition of our dark theme:
.theme-dark body {
    --background: black;
    --color: white;
}
```

This is how we define our themes and their props. It's that simple! It's the act of defining CSS variables that we'll be using throughout our app. We usually target the **body** element that's inside a `theme-xxx` class.

> **Note:** The *target* of a theme (in this case, the **body** element) can, of course, be anything. As we'll see later when we talk about multiple theme categories, we'll be using other targets.

Next, we're making it so that clicking the button toggles between our two defined themes. This is easy! We just add the new theme's CSS class name and remove the previous one:

```
var currentTheme = 'theme-light';

button.addEventListener('click', () => {
  // We're just toggling between 2 themes here: 'theme-light' and 'theme-d
```

```
// We're just toggling between 2 themes here: 'theme-light' and 'theme-d
ark'

  var prevTheme = currentTheme;
  currentTheme = currentTheme == 'theme-light' ? 'theme-dark' : 'theme-lig
ht';

  // Remove the previous theme from the html class list
  document.documentElement.classList.remove(prevTheme);

  // Add the new theme name to the html class list
  document.documentElement.classList.add(currentTheme);
});
```

All that's left is to use the theme props we defined wherever we need them:

```
.root {
    background: var(--background);
    color: var(--color);
}
```

This is a simplified but functioning theming system! Everything else builds on top of this, and while an actual implementation might be more complex, the basic idea of how it works is shown in this example.

We already have a working theme switcher in just a few lines of code! But a theme is much more than this. Starting in the next lesson, we'll be talking about all the additional components of our themes. Let's continue!

Backgrounds and Foregrounds

This lesson introduces the concept of backgrounds and foregrounds in our theming platform.

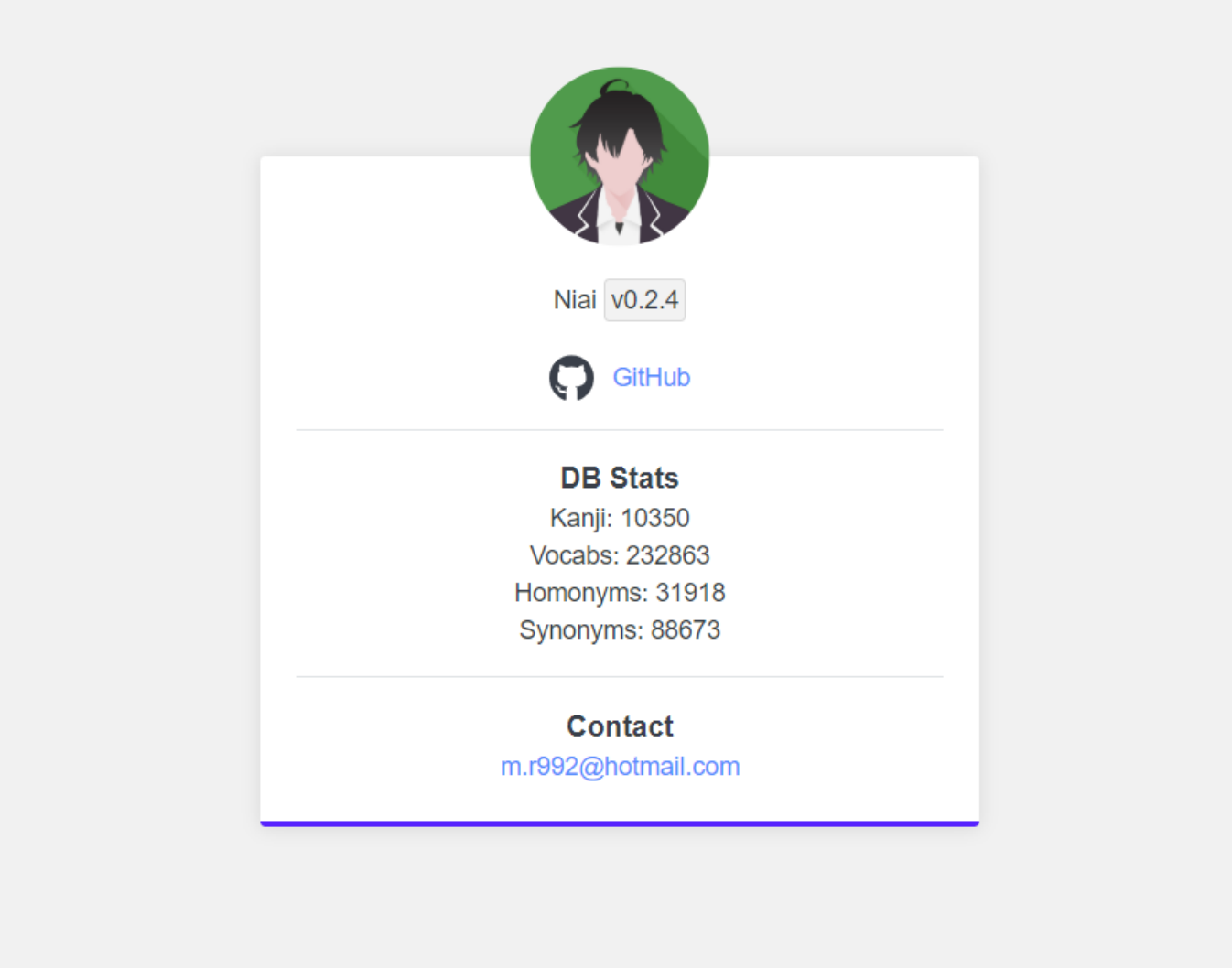
WE'LL COVER THE FOLLOWING

- Backgrounds
- Foregrounds
- Real-world example

Backgrounds

Most apps have some kind of main background color. By default, that's white in the browser. Lots of apps choose to select a grey color so that cards can be displayed on top of it. This makes for a good aesthetic view.

Let's have a look at the following page:



This color is what we call the main background, `--bg-main`, of the app.

Foregrounds

To put it simply, a *foreground* is also a background color, unlike some UI frameworks that call text colors a “foreground”. We call it such because those colors are used in the foreground of the app, for example in cards.

In the following picture, the cards use the main foreground color (white in this case), and behind them is the main background color (grey).

Let’s focus on a single card for a second:

The arrows point to elements with a darker foreground to contrast them a bit. This is common, but we won’t introduce another name for these, because this is simply another foreground color. That’s why we’ll define multiple foreground colors in rising levels. Each level is a darker color than the one before it, i.e., each level darkens the main foreground color even more. In the app we’re taking these screenshots from, it appears we only need 2 foreground colors: white and the slightly grey color.

The main foreground color is always called `--fg-0` (white), so we’ll assign `--fg-1` to the grey color.

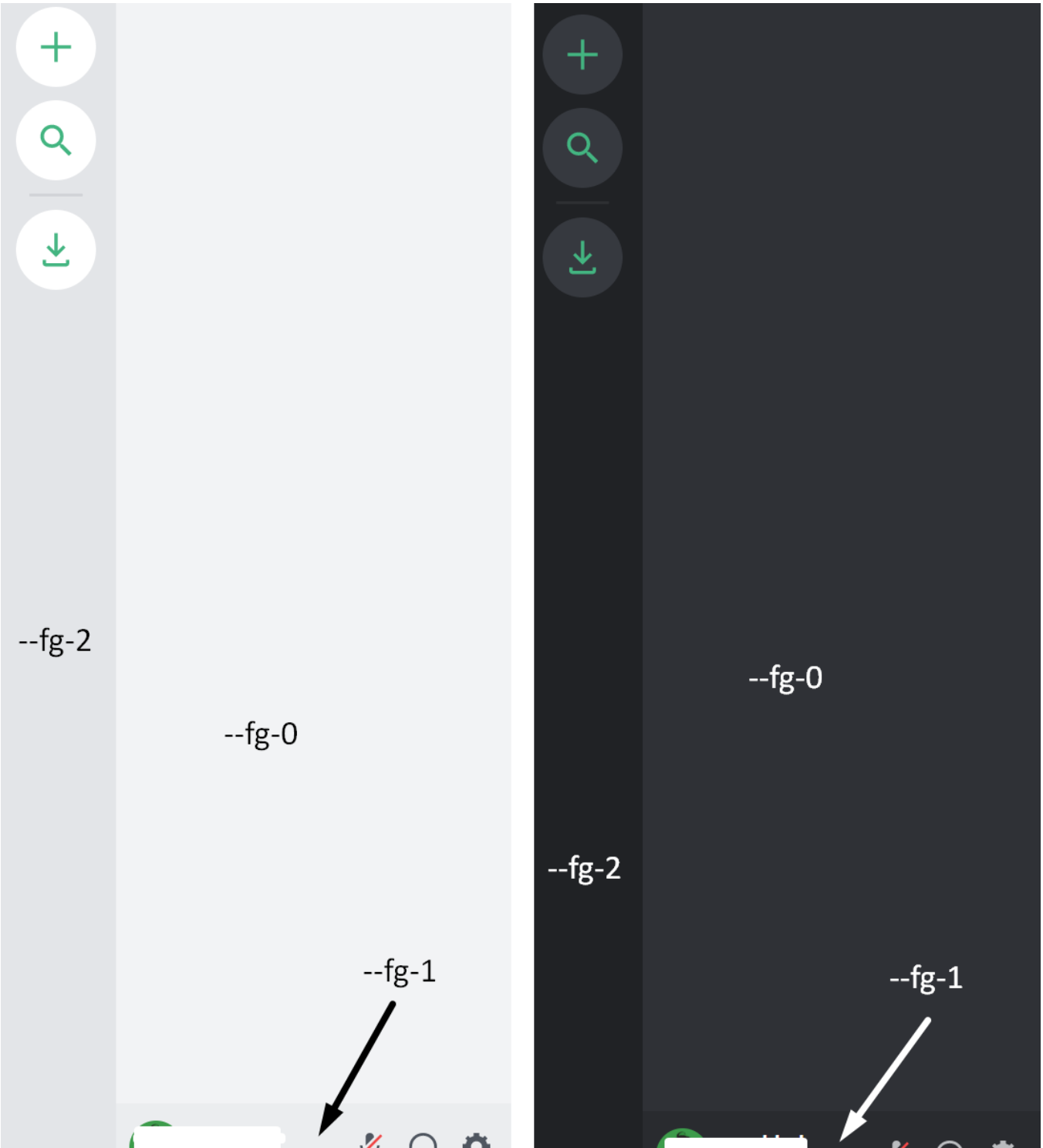
In a dark theme:

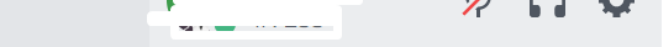
The variables for foregrounds follow the same pattern but using dark colors. So when developing our app, we only worry about what level we want to use, and then we can be sure our design will be pretty no matter the theme.

In other apps, we might need more levels and that's fine. Using our theming platform, we should be able to customize how many we want.

Real-world example

Let's take an example from discord, where we can see how we'll need at least 3 levels:





This shows how easy consuming these variables can become.

The next lesson gives details on palettes, swatches, and color schemes.

Palettes, Swatches, and Color Schemes

This lesson goes into what palettes, swatches, and color schemes mean in our platform and how defining these will make it much easier to customize our apps.

WE'LL COVER THE FOLLOWING

- Palettes
- Swatches
- Color schemes

Palettes

A color palette is a collection of colors we might want to use.

It's important to prepare a palette as this makes our usage of colors consistent throughout the app.

Swatches

A swatch is a sample of a color chosen from a range of similar colors. It's very common to use several swatches of a color in different scenarios.

For each swatch, there's a CSS variable. This is how we'll name our swatches: `--{color-name}-{swatch}`

The `{swatch}` is a number, by default **50**, **75**, **100**, **200**, **300**, **400**, or **500**.

This gives us variables for a single color (for example, for the `blue` color):

```
--blue-50: #e3edff;
--blue-75: #c7dcff;
--blue-100: #b4d0ff;
--blue-200: #a2c5ff;
--blue-300: #8fb9ff;
--blue-400: #69a1ff;
--blue-500: #448aff;
```

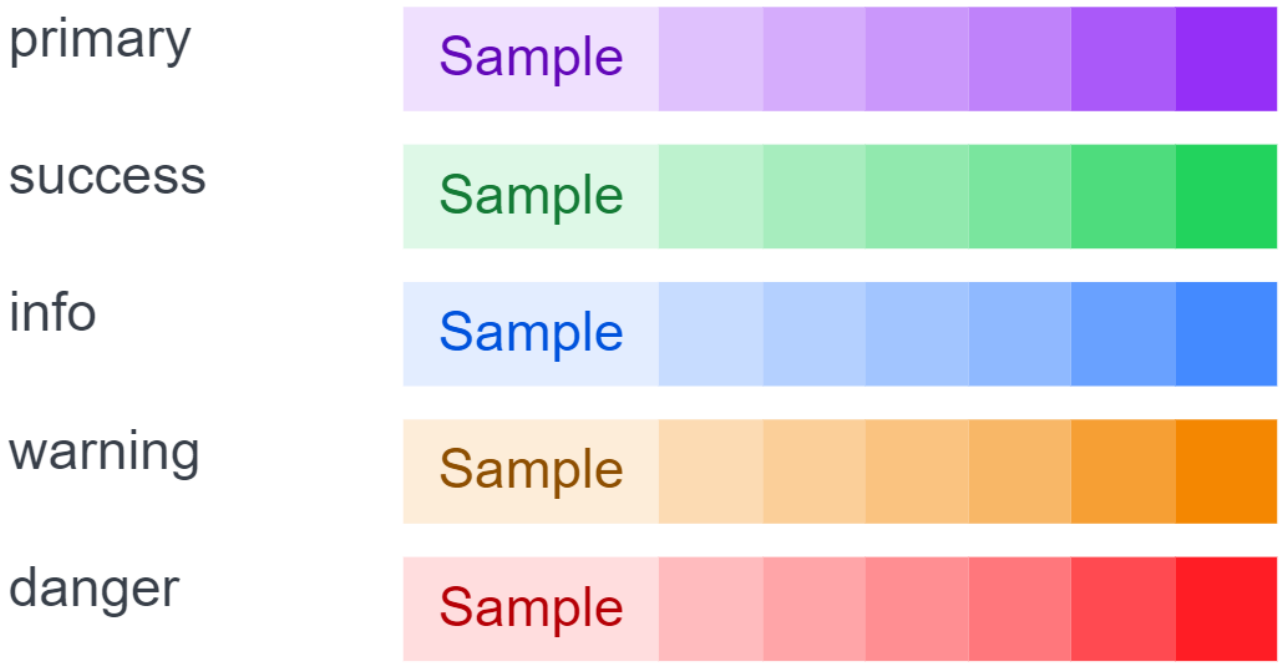
> **Note:** Sometimes, we might need a blue to use as the text color. Usually, using `--blue-500` won't be good enough, so we'll define an additional `--{color-name}-text` variable for each color to be used exclusively for texts. We'll learn more about this in a later lesson.

These are the default swatches, but in a later section you'll learn how you can customize this in `css-theming`.

Color schemes

Choosing a color scheme involves handpicking certain colors and giving them a **meaning** in our app. An example of this is picking a *blue* and using that as the *primary* color of our app. We call this a **semantic color** when we refer to it, as it visualizes a meaning in the app instead of a raw color.

Here's an example of a color scheme:



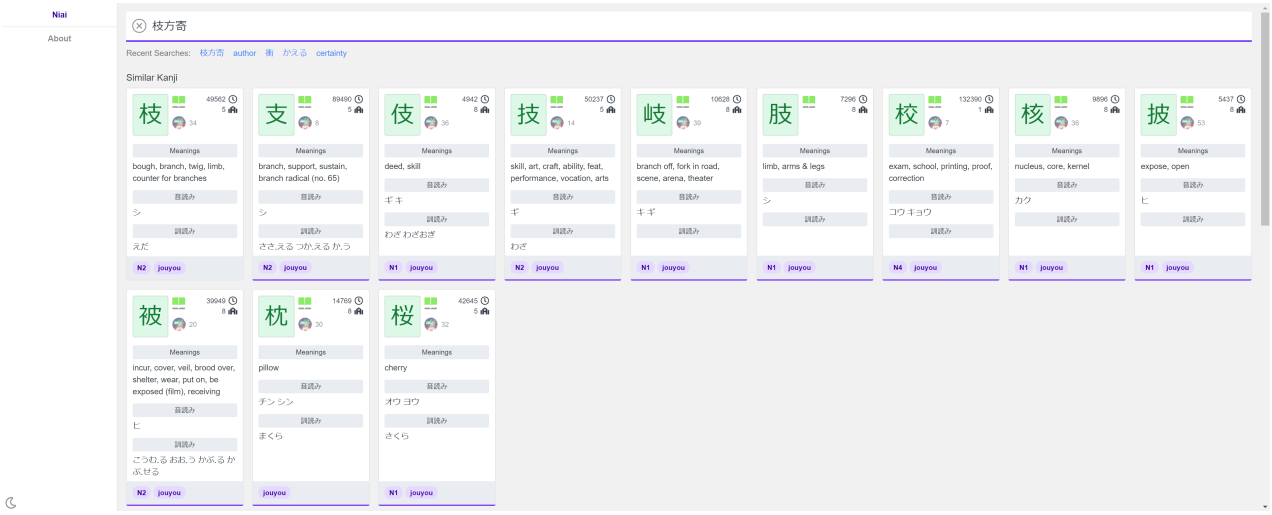
In this example, and from what we see in the figure, we have decided that we'll use **purple** as the color for **primary**. Assuming we use **primary** as the accent in our app, this means we'll be seeing **purple** in those places. The same applies to the other semantic colors.

> **Note:** Semantic colors are not set in stone, we should be able to add to these and change them to whatever we think is useful in our app.

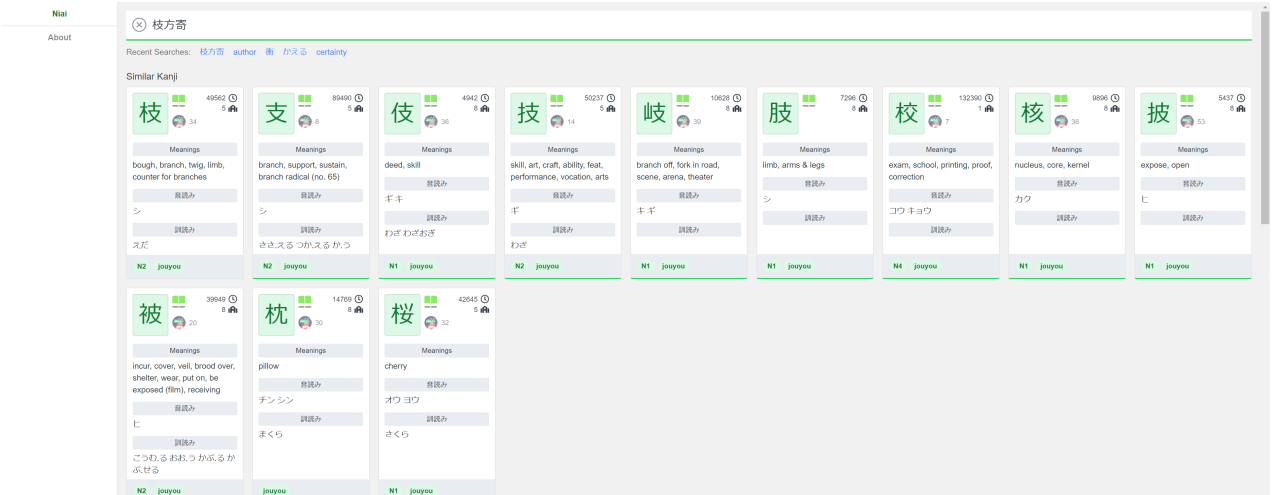
Defining a color scheme is liberating when developing components, as we don't have to think about exact colors, we simply choose a semantic color that has the appropriate meaning.

This also makes it easier to create a different theme that updates the whole feel of the app simply by changing a semantic color. Below we're changing the **primary** semantic color to show you the effect:

- Indigo as the `primary` color:



- Green as the `primary` color:



That’s how palettes, swatches and color schemes provide us with wide-range flexibility to design a UI. The next lesson explains what theme brightness is.

Theme's Brightness

This lesson introduces the concept of a theme brightness.

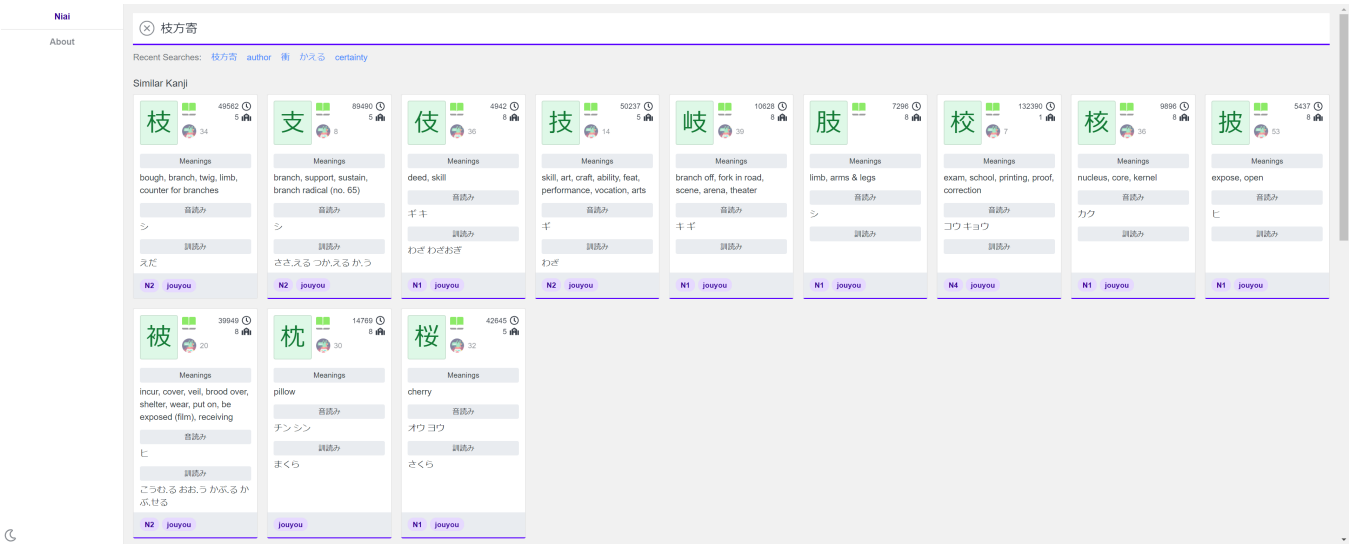
WE'LL COVER THE FOLLOWING

- Introduction
- Explanation

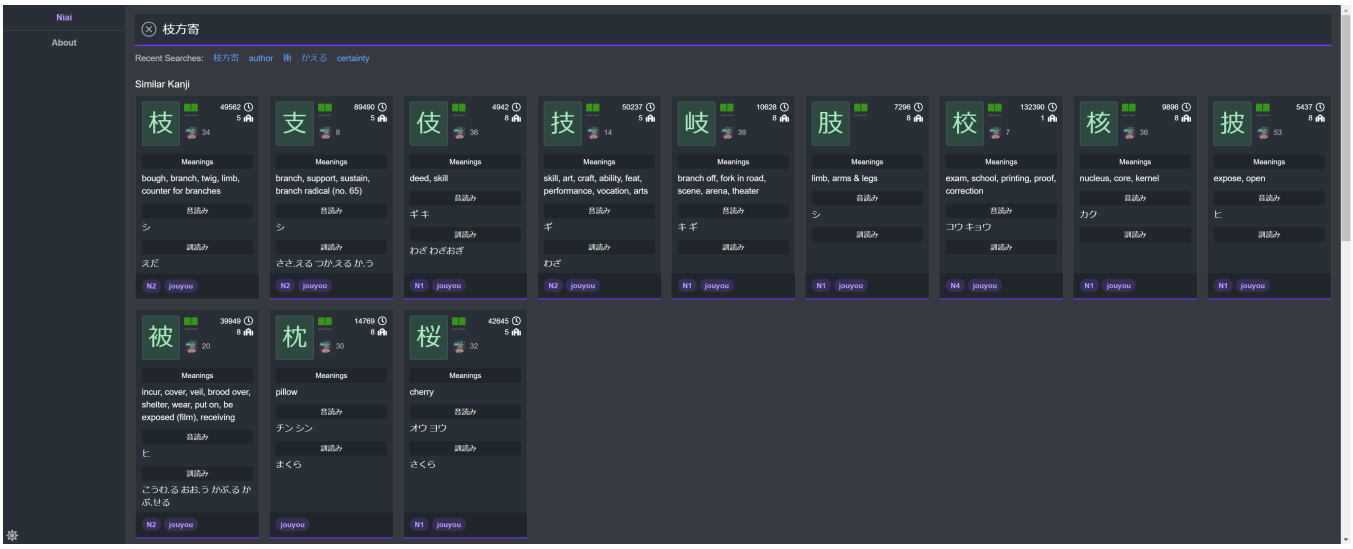
Introduction

A **theme's brightness** defines the *mood* of a theme. For now, we'll assume there are only two brightness levels:

- Light



- Dark



Having a dark theme is becoming more and more popular these days, so I'm sure you already expected a CSS theming course to talk about exactly that! But why do we have to talk about it? I mean, isn't a dark theme just like any other theme? Why should we treat it differently?

Explanation

This comes around to human biology really, but this would be the wrong course for that! In short, there are certain things in the difference between a light and a dark theme that forces us to treat it especially, in particular when it comes to colors (which we'll talk about in the next lesson). Every brightness is associated with a certain basis for backgrounds/foregrounds in our app. A light theme always goes with lighter colors, a dark theme always goes with darker colors.

Whenever we talk about a certain theme, we should always know its brightness, as that will determine how we'll compute certain values/colors in the theme. In the next lesson, you'll learn about the problems related to colors, and what it means to mix them.

Mixing Colors

This lesson discusses the main problems that arise when dealing with colors in an application and how mixing colors in a certain way provides the solution.

WE'LL COVER THE FOLLOWING

- Problems
- Solutions
  - Using the alpha channel
  - Mixing colors
  - Dynamic mixing

Problems

Here's our conundrum when it comes to colors:

- Manually picking up the different variations of colors we need in our app is hard and prone to error.
- When different themes have different brightness levels, the color from one of them usually doesn't translate very well to the other.

Here's an example of what we're talking about. Let's create some `label` and use *blue* as a `background` and `color`, we'll use a lighter blue for the background:

Output

HTML

CSS (SCSS)

```
This is a label: css-theming
```

Light mode

Now, what if we're in a dark theme:

Output

HTML

CSS (SCSS)

```
This is a label: css-theming
```

Dark mode

It becomes obvious that the light blue background is out of place in this dark theme. It's a *dark* theme, so why are we using a color that's very close to white as a background?

Solutions

Using the alpha channel

The alpha channel is the 4th component of an RGB**A** color. It controls the *opacity* of the color. A **0** alpha means the color is fully invisible, while **255** means it's fully opaque.

So, let's talk about a better way to color this `label` component. A straightforward solution is to use a blue color with an alpha channel.

Output

HTML

CSS (SCSS)

```
This is a label:  css-theming
```

Light mode

Output

HTML

CSS (SCSS)

```
This is a label: css-theming
```

Dark mode

> **Note:** Let's ignore for now that the text itself looks bad. We'll talk specifically about text colors and how to correct them in a later lesson.

Sure, that's a bit better than having a whitish color in a dark theme throwing our concentration off. But this has several problems; first, a side effect of using an alpha channel, the background color of the label will blend into the background color behind it. This is known as **Alpha compositing** and can create all sorts of visual nuisances, the result will vary depending on the background color as well:

Output

HTML

CSS (SCSS)

```
This is a label:  css-theming
```

Dark mode

The result is almost unreadable, and certainly fails the AA standard contrast ratio! Try it with any other extreme color, just change the `background` property of the `body` element.

The problem here, as we stated, is that the transparent color we chose for the `label` is blending with the background color of the `body` behind it, thus creating a different color than what we intended. This is almost always bad for colors used as backgrounds.

And so, we reach the next solution.

Mixing colors

Let's imagine that we magically found the following 2 colors:

- `#e3edff`: We were told to use this in light themes.
- `#324055`: We were told to use this in dark themes. It might not look like it on a white background, but this is actually a blue color.

Let's try and see:

Output

HTML

CSS (SCSS)

```
This is a label: css-theming
```

Light mode

Output

HTML

CSS (SCSS)

```
This is a label: css-theming
```

Dark mode

This is exactly what we want! It even works in any background because `#e3edff` (background color of the `label`) is a fully opaque color:

Output

HTML

CSS (SCSS)

```
This is a label: css-theming
```

Dark mode

> **Note**: Don't worry that the text color in the above examples was changed too, we'll talk about this extensively in the next lesson. For now, we're focusing on the background color.

Do note that the examples here are extremely simplified. In a real-world app, the problems mentioned will be more catastrophic, and updating your app to use the correct approach of mixing colors can certainly get you much closer to a good looking app in any brightness.

Dynamic mixing

Now, we imagined that the *correct* colors magically appeared in front of us. That's fine for this course, but how do we obtain those colors in a real app? Handpicking them? That would be a nightmare!

SCSS can help here once again. We'll take advantage of the `mix` function in SCSS:

```
mix(color1, color2, percentage)
```

Output

JavaScript

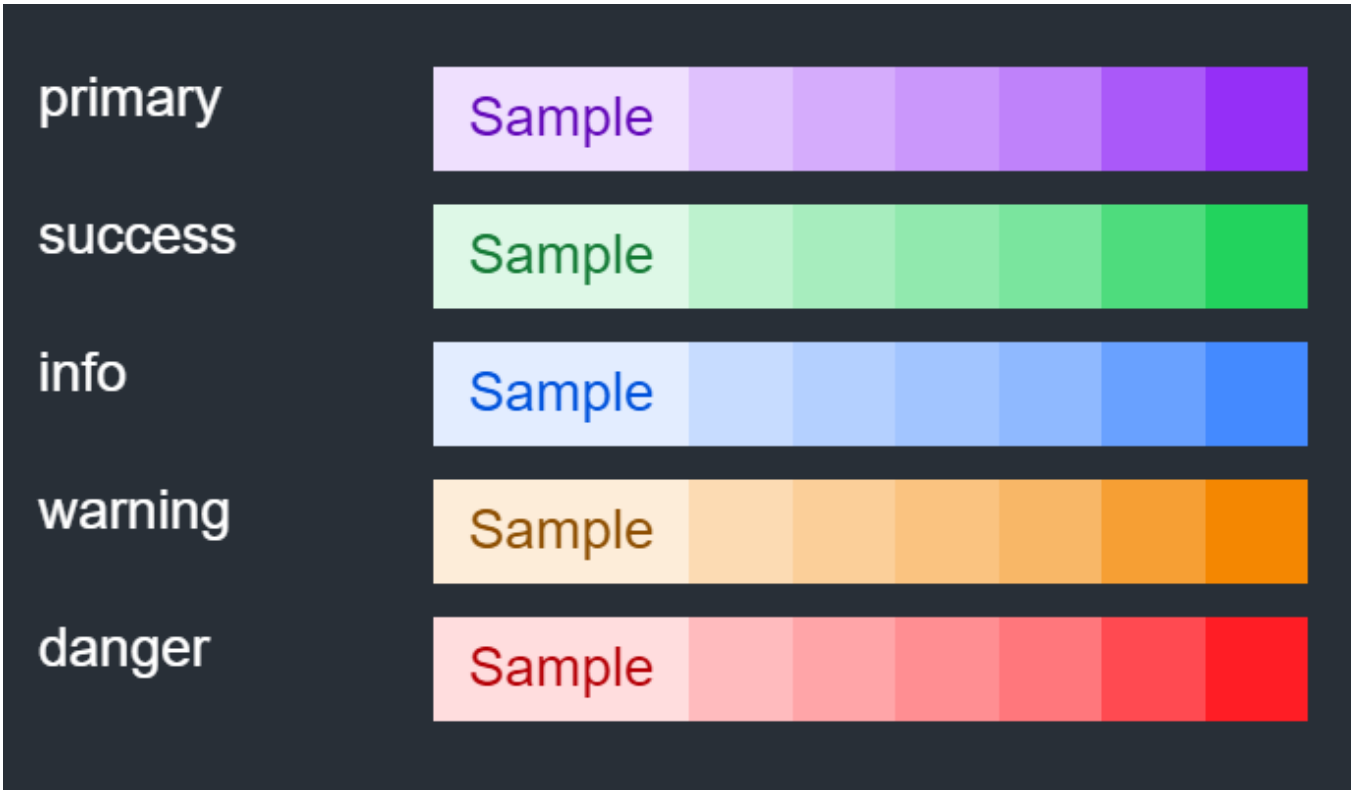
HTML

CSS (SCSS)

Light mode

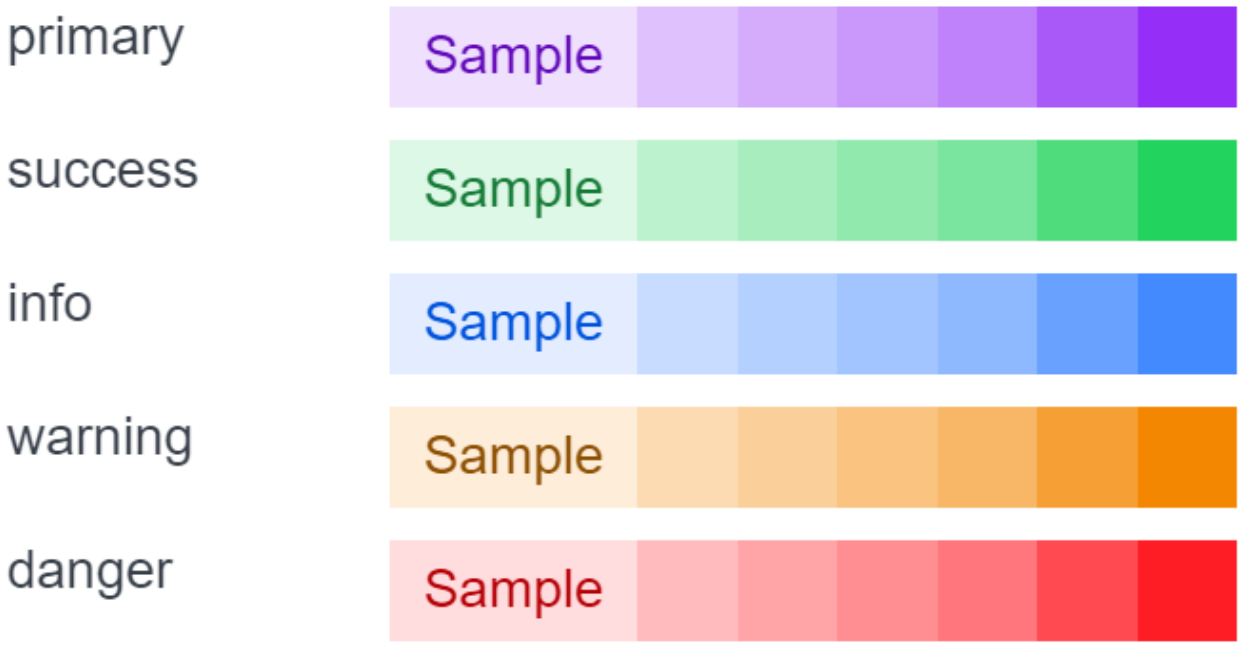
In the above example, we were able to easily recreate fully opaque colors that could act as backgrounds for light and dark themes. Using this pattern, we can recreate the whole palette and create versions that are appropriate for their respective theme brightness.

Instead of this palette that isn't really appropriate in a dark theme:

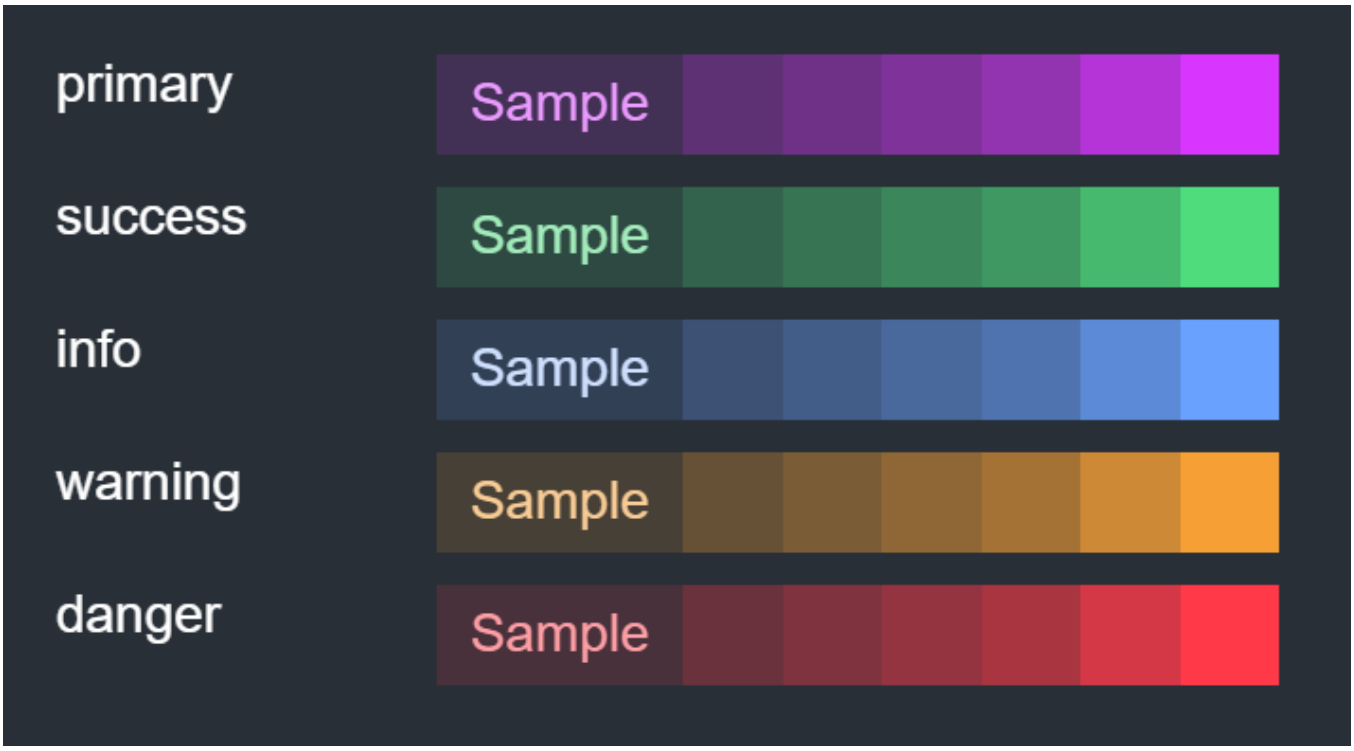


The result ends up being the following:

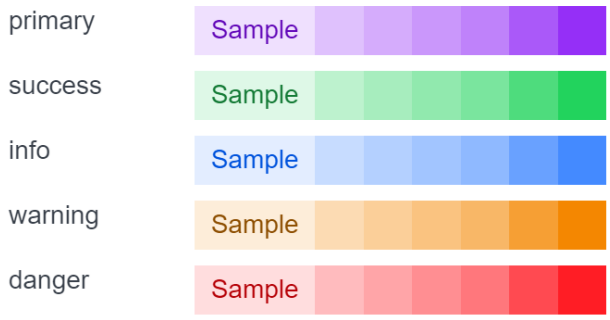
- Light:



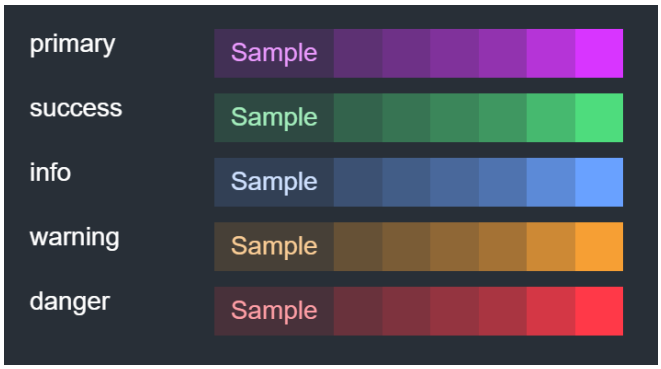
- Dark:



Dynamic mixing in a light theme:



Dynamic mixing in a dark theme:



That's how we can properly mix colors for use in backgrounds in any theme. But, we still encountered problems with the text. The next lesson focuses on text colors.

Text Color

This lesson talks about the text color in different themes.

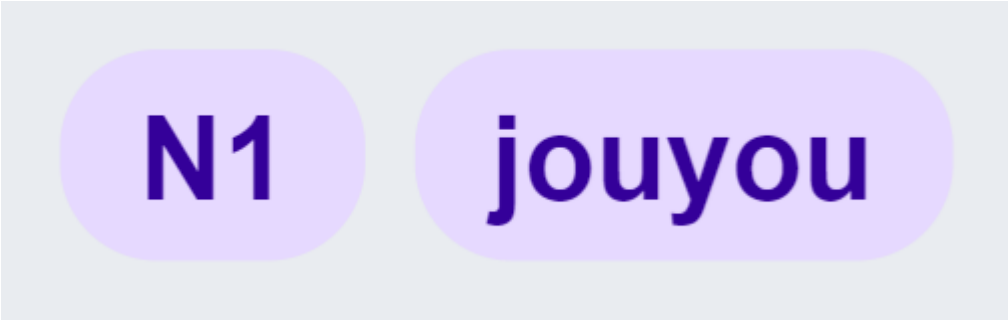
WE'LL COVER THE FOLLOWING

- Main text colors
- Palette text colors
- Tools
- Conclusion

Text colors, for the most part, are easier to deal with than background colors. The “Alpha compositing” problem that we discussed in the previous lesson doesn’t apply as severely here, and we usually only need one or two main text colors: a text color and a secondary text color. An example use case of this is using the secondary text color for less emphasized text:

```
This is a title
This is a subheading
```

In addition to the main text colors, we might also need a text color for each color in our palette. A nice use case for this is creating colorful labels:



As you can see, we’re using a purple text color over a light purple background. If you recall, the CSS variable for the text color here would be `--purple-text`.

Main text colors

Main text colors

We can start by defining the main text color in the app, which will depend on the brightness of the theme.

For light themes, we'll use a blackish color and for dark themes, we'll use a whitish color. To create a secondary text color that we can use in places where you want less emphasis on the text, we can use opacity (alpha channel) on the main text color. For that we'll be using `rgba`:

```
rgba($text-color, .7)
```

The text color blending into the background here is often desirable (unlike with background colors, as explained in the previous lesson), so using the alpha channel is usually OK for text color.

- Light theme

Output

HTML

CSS (SCSS)

```
This is a title
This is a subheading
```

- Dark theme

Output

HTML

CSS (SCSS)

```
This is a title
This is a subheading
```

Palette text colors

Sometimes, we might want to style a certain component with a palette color as a background. Let's take a label as an example:

Output

HTML

CSS (SCSS)

```
css-theming
```

This works, but a nicer look is using a similar color to the background as a text color:

Output

HTML

CSS (SCSS)

```
css-theming
```

This is not bad, but because we're using the same color in the background, we might want to see a bit more contrast by darkening the text color. We do this in SCSS using the `darken` function:

```
darken(color, percentage)
```

> **Note:** darken is a SCSS function that returns a darker color than the one you give it, according to the percentage you specify.

Output

HTML

CSS (SCSS)

`css-theming`

Now it looks better, and it has become more compatible with AA standard contrast ratios.

This does it for the light theme. In a dark theme, we'll have to do the inverse. Let's first display an example from the previous lesson where the normal color of blue is used in the text and a dark-compatible color (`black`) is used as a background:

Output

HTML

CSS (SCSS)

```
css-theming
```

In the case of a dark theme, we'll have to `lighten` the text color a bit to get a good result:

```
lighten(color, percentage)
```

> **Note:** lighten is a SCSS function that returns a lighter color than the one you give it, according to the percentage you specify.

We'll get a *paler* text color which goes well with the overall dark nature of the theme:

Output

HTML

CSS (SCSS)

```
css-theming
```

Tools

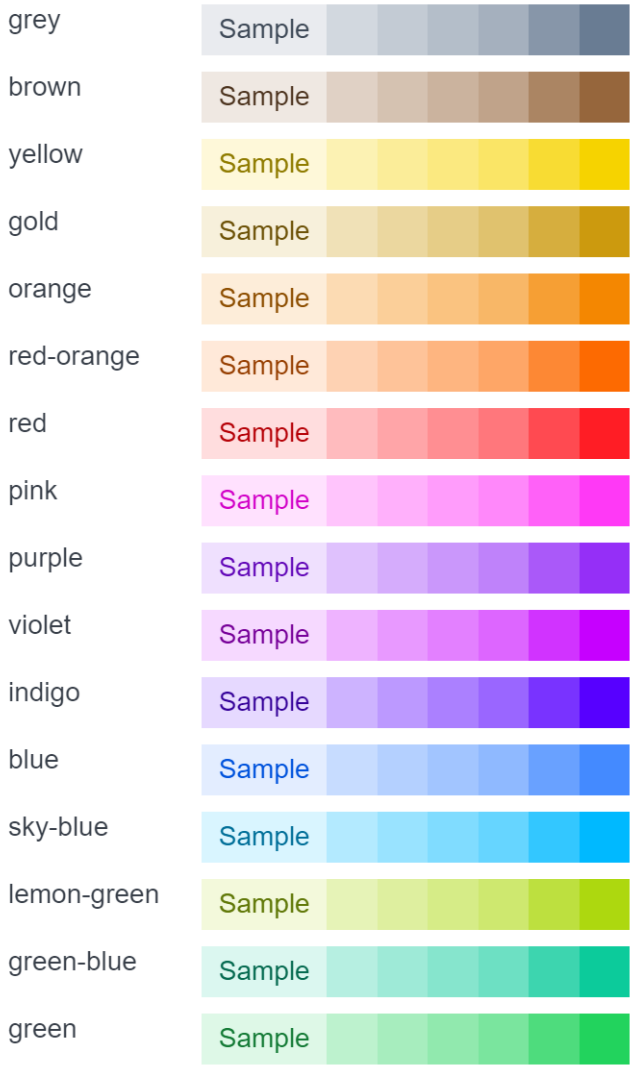
Here are some useful tools that can tell you how close to standard contrast your colors are:

- colorable
- contrast-ratio

Conclusion

Using this method of mixing and creating colors will result in the following palette:

- Light theme



- Dark theme

That's pretty much it for text colors! In the next lesson, we'll learn how to deal with borders.

Borders

This lesson discusses how we'll deal with borders in our app.

WE'LL COVER THE FOLLOWING

- Using alpha channel
- Borders around cards

Borders are commonly used in most apps. This is one of the easier topics to talk about. We'll be talking about only two variables for borders:

- `--bd`: The normal border color.
- `--bd-focus`: The more stressed border color can be used on hover, for example.

Using alpha channel

We'll use an alpha channel with either white or black to create our borders. We'll be using `rgba` again here:

- Light theme

```
// We're aiming for a simple transparent black border, so we set the alph
a channel of black:
$bd-light: rgba(0, 0, 0, 0.1);
$bd-focus-light: rgba(0, 0, 0, 0.2);
```

- Dark theme

```
// We're aiming for a simple transparent white border, so we set the alph
a channel of white:
$bd-dark: rgba(255, 255, 255, 0.1);
$bd-focus-dark: rgba(255, 255, 255, 0.2);
```

Some simple usages:

Some simple usages:

Output

HTML

CSS (SCSS)

First
Second
Third

Borders around cards

技

50237

5

14

Meanings

skill, art, craft, ability, feat, performance, vocation, arts

音読み

ギ

訓読み

わざ

N2 jouyou

Since our borders have an alpha channel, they'll work pretty much anywhere. The only decision we have to make is whether to use `--bd` or the more stressed `--bd-focus`.

> **Node:** Changing these variables to point to the correct light or dark version is, once again, the responsibility of the theming platform. You get that for free when using `css-theming`.

That's all about borders, in the next lesson we'll learn how icons and svgs fit into our theming platform.

Icons and Svgs

This lesson discusses how we'll make icons and svgs play nice with our theme.

WE'LL COVER THE FOLLOWING

- Font icons
- Svgs

Font icons

Font icons are the same as any text, so we don't have much to talk about here. Simply changing the color of the text will change the color of the icons:

- Light theme

- Dark

Svgs

An **svg** is another type of HTML element that helps us to manipulate and interact with different animated graphics and shapes.

That is why they need some special attention. Unlike normal HTML elements, svgs aren't effected by `background` and `color` CSS properties. We use `fill`, `stroke`, and `stroke-width` to control the color. Let's quickly show what those properties control.

Here's an svg that has a fill:

Here's an svg that has a fill:

Output

HTML

CSS (SCSS)

Here's an svg that has a stroke:

Output

HTML

CSS (SCSS)

Here's an svg that has both a fill and a stroke:

Output

HTML

CSS (SCSS)

Usually, we'll be using ready-made svgs we find on the web or ones that are given to us by a designer. For most of those svgs they'll only have a `fill`. We want simple svgs to act the same as text, meaning we want to color them as if they were text. We can do the following:

```
svg {
  fill: currentColor;
}
```

We're setting the `fill` property to `currentColor`, instructing the `fill` color to get its value from the currently inherited `color` value.

> **Note:** This way of styling the svgs only works with inline svgs.

Since we're styling the svg, we can do whatever we want if it has a stroke too:

```
svg {
  // You could also use currentColor for the stroke and something else fo
r the fill if it has a fill.
  stroke: var(--primary);
}
```

That's it for icons and svgs. In the next lesson, you'll learn how to get programmatic access to design values.

Programmatic Access

This lesson discusses how to have programmatic access to all the data we define in CSS.

WE'LL COVER THE FOLLOWING

- What is programmatic access?
- Explanation

What is programmatic access?

As developers who use web technologies, we're used to working across three different but cooperative languages: HTML (the layout), JS (the script), CSS (the style).

Sometimes, it's beneficial to be able to access what we define in CSS from the script. This is what we mean by gaining programmatic access to the values we define in our CSS/SCSS.

Let's look at a use case. Assuming we have 3 different themes in our app, we'll have those defined somewhere in our CSS/SCSS files. It's a common need to display some kind of page where the end user can select what theme they want. In such a case, we'll want a way for our code to get the names of those themes. We *could* put the theme names in our code file in addition to their definitions in CSS/SCSS, but that would be redundant and will double the places we'll have to edit if we ever change anything.

What we need is a way to obtain the different CSS values we're defining in CSS programmatically.

Explanation

In JavaScript, we do this by obtaining the computed styles of an element:

```
const style = getComputedStyle(element);
```

```
const value = style.getPropertyValue(prop);
```

The `prop` above can be any CSS prop or CSS variables. Our strategy here is to somehow put the theme names in a CSS variable.

Let's assume we have our themes defined as a SCSS `map`:

```
$themes: (
  'light-theme': (...),
  'dark-theme': (...)
);
```

To get a list of the theme names we'll map the keys to obtain a list of the names:

```
$theme-names: map-keys($themes);
```

Then, put the result in a CSS variable so that we can obtain it later in code:

```
body {
  --theme-names: #{$theme-names};
}
```

The `--theme-names` variable will contain a comma delimited list of the names now.

Then, obtaining those names in code becomes easy:

```
const style = getComputedStyle(document.body);
const names = style.getPropertyValue('--theme-names');
const themeNames = names.split(',').join(', ');
```

Putting all of this together:

Output
JavaScript
HTML
CSS (SCSS)

Using this method, we'll be able to gain access to most things we define in CSS. So that even when we get to a point where we have a platform that's fully customizable, we'll easily be able to obtain all the values dynamically and keep a single source of truth. We'll be able to create a page that's dynamic and changes with the definitions in our CSS.

Here's an example of such a page, taken from the basic sample in `css-theming`:

Themes

default light default-dark dark

Palette

grey	Sample	primary	Sample
brown	Sample	success	Sample
yellow	Sample	info	Sample
gold	Sample	warning	Sample
orange	Sample	danger	Sample
red-orange	Sample		
red	Sample		
pink	Sample		
purple	Sample		
violet	Sample		
indigo	Sample		
blue	Sample		
sky-blue	Sample		
lemon-green	Sample		
green-blue	Sample		
green	Sample		

With this, having access to all the design values programmatically is possible without any redundancy. Now, let's move on to discuss the different types of themes.

System and User Defined Themes

This lesson talks about the different types of themes in an app.

WE'LL COVER THE FOLLOWING

- System defined themes
  - Example
- User defined themes

System defined themes

A theme that is defined by the developers of the app is called a **system defined theme**. In the simplest case, an app would have two defined themes: a *light* and a *dark* theme.

A system defined theme's props are defined by the designer of the app.

Example

Assume that we have a `border-radius` value we use in cards in the app. Right now, it's a constant value and remains the same no matter the currently applied theme. But let's say we want it to change per theme so that we can tweak it between different themes. Our theming platform will be able to support this.

User defined themes

We talked before about *system defined themes*, and how they're defined by the developers of the app. This means the app will have a fixed number of well-known themes that the user can choose from. Imagine that we want to allow the user to customize certain (or all) aspects of the theme; this is what we call *user defined themes*.

We learned in a previous lesson that we can access design values (CSS props)

programmatically:

```
const style = getComputedStyle(element);
const value = style.getPropertyValue(prop);
```

In a similar way, we can set those values:

```
const style = getComputedStyle(element);
const value = style.setPropertyValue(prop, value);
```

In theory, this allows the user to not only choose one of our system defined themes, but even create one of their own and customize the props of the theme to whatever they like.

Now that you have basic insights on the general theming concepts, let's begin using all the ideas we introduced by learning how they fit in the `css-theming` package.

Conclusion

A few final words.

WE'LL COVER THE FOLLOWING

- The `css-theming` package
- Feedback

So far, we've discussed a multitude of problems and solutions when it comes to CSS theming. Most of these problems aren't hard to solve but having a single platform that solves them and presents the notions in a clear way can make a big difference in productivity.

The `css-theming` package

This is where `css-theming` comes into play. `css-theming` solves all that we've discussed and more. The `css-theming` package is open sourced in this github repo.

In the next section, we'll learn how `css-theming` puts all of those notions together in one easy to use package!

Feedback

I'll be looking into improving this course and adding more details to it with time as I discover new things that need explaining. I might have missed some use case or forgot to talk about something. If you have any questions, suggestions, or feedback please reach out to me on twitter or by email!

Installation

This lesson shows how to install the css-theming package.

WE'LL COVER THE FOLLOWING

- Installing `css-theming`
- Modules
- JS
- SCSS

Installing `css-theming`

The `css-theming` package is released as an npm package.

npm v1.2.0

Type the following in your favorite terminal to install it:

```
$ npm i css-theming --save
```

Modules

There are both JavaScript modules (typescript type definitions available) and SCSS src files after you install the package.

JS

You use the JavaScript modules the usual way:

```
import { initializeTheming, ... } from 'css-theming';
```

SCSS

The SCSS src files are at `css-theming/src/scss/`:

```
// Change "../" depending on where your file is.
@import '../node_modules/css-theming/src/scss/css-theming';
```

> **Note:** Check the different samples in the GitHub repo to see this up close.

After you've successfully installed `css-theming`, move to the next lesson to begin the setup.

Setup

This lesson shows how to do a basic setup of css-theming in your app.

WE'LL COVER THE FOLLOWING

- Defining system themes
- Importing the main `css-theming` SCSS file
- Calling `initializeTheming`
- Setting a new theme

Following are three things you have to do to setup your project to work with `css-theming`:

- Optionally define themes in your variables file.
- Import the main `css-theming` SCSS file from your app scss file.
- Call `initializeTheming` from javascript/typescript in the startup of your app.

Defining system themes

By default, `css-theming` gives you two themes, one *light* theme called `default`, and another *dark* theme called `default-dark`. You're free to completely override this to define your own system themes.

You override this in your **variables** file. A **variables** file is a SCSS file you use that doesn't emit any real CSS. It contains all the definitions (SCSS variables, mixins, functions, etc.,) of your app. You usually import this file in all your concrete SCSS files; those would be the main app SCSS file, and SCSS files for components if your app is a SPA.

The following is an example **variables** file that defines three themes:

```
// Override the system defined themes
```

```
$ct-themes: (
  'default': (
    'brightness': 'light',
  ),
  'default-dark': (
    'brightness': 'dark',
  ),
  'high-contrast': (
    'brightness': 'light',
    ...
  )
);

@import '../../node_modules/css-theming/src/scss/pure';
```

We import the **pure** SCSS file from `css-theming` after we override the variables we want, as shown in the above snippet. This **pure** file is what contains all the definitions of `css-theming`, including all SCSS variables it defines (themes, colors, swatches, etc.).

> **Note:** You can think of this **pure** file as the **variables** file of `css-theming`.

What we're doing here is almost exactly the same as what you'd do when using something like bootstrap for example. You would import bootstrap's **variables** file in all your SCSS files, but you only import bootstrap's main file in your main app SCSS file.

Importing the main `css-theming` SCSS file

In your main app SCSS file:

```
@import '../../node_modules/css-theming/src/scss/css-theming';
```

Calling `initializeTheming`

Call `initializeTheming` as soon as you can in your app. This can be the `created` lifecycle hook of the main app component in a Vue.js app, the constructor of the main app component in an angular app, etc.

```
import {
```

```
  initializeTheming,
} from 'css-theming';

// Initializes the default category themes
initializeTheming(/* theme */);
```

In this basic example, `initializeTheming` can take the theme that you want to initialize at first. Let's assume we always want to load `default-dark` whenever the application loads:

```
import {
  initializeTheming,
  getTheme,
} from 'css-theming';

// Initializes the default category themes
initializeTheming(getTheme('default-dark'));
```

Of course, instead of hard coding the initial theme you load, you can easily get this value from local storage or from your backend if you save the user's preference there.

> **Note:** If you register additional category themes you'll call `initializeTheming` once for each category, as we'll explain in a later lesson.

Setting a new theme

You tell `css-theming` to change the current theme by calling `setTheme`:

```
import {
  setTheme,
} from 'css-theming';

// Initializes the default category themes
setTheme('[theme-name]');
```

This of course can be in response to a button click for example, that the user clicks on to choose what theme they want to activate.

After we've successfully setup our app to use `css-theming`, we can learn about

After we've successfully setup our app to use `css-theming`, we can learn about the different ways we can customize it. Please move to the next lesson to see how it's done.

Customizing System Themes

This lesson shows how to override the default system themes definition.

Customizing the default system themes involves defining the `$ct-themes` map in your **variables** file. Here's the *default* definition inside `css-theming`:

```
$ct-themes: (
  'default': (
    'brightness': 'light',
    // 'colors': overrides $ct-colors-[light|dark]
    // 'semantic-colors': overrides $ct-semantic-colors
    // 'font': overrides $ct-font
    // 'text-color': overrides $ct-text-color-[light|dark]
    // 'bg-main': overrides $ct-bg-main-[light|dark]
    // 'fg-target': overrides $ct-fg-target-[light|dark]
    // 'fg-swatches': overrides $ct-fg-swatches-[light|dark]
    // 'bg': overrides $ct-bg-[light|dark]
    // 'bg-focus': overrides $ct-bg-focus-[light|dark]
    // 'bd': overrides $ct-bd-[light|dark]
    // 'bd-focus': overrides $ct-bd-focus-[light|dark]
  ),
  'default-dark': (
    'brightness': 'dark',
  ),
) !default;
```

Some of the theme props are well known by `css-theming` and it'll try to fall back to defaults when you don't override them. All those props are mentioned in the code snippet above as comments. The only required prop is `brightness`.

In addition to those well-known props, you can add whatever prop you want to change between themes. We'll discuss this later. Next, let's focus on customizing colors.

Customizing Colors

This lesson discusses how to customize and override colors and their swatches.

WE'LL COVER THE FOLLOWING

- Customizing main colors
- Customizing semantic colors
- Customizing swatches

Customizing main colors

By default, `css-theming` will fall back to the following pre-defined colors map for each brightness:

```
// Colors used in light themes
$ct-colors-light: (
  'grey': #697C93,
  'brown': #96663C,
  'yellow': #F6D300,
  'gold': #CC9A0E,
  'orange': #F48701,
  'red-orange': #FD6A01,
  'red': #FF1D25,
  'pink': #FF39F6,
  'purple': #952FF7,
  'violet': #C600FF,
  'indigo': #5700FF,
  'blue': #448AFF,
  'sky-blue': #00B9FF,
  'lemon-green': #ADD80F,
  'green-blue': #0CCB9B,
  'green': #22D35D,
) !default;

// Colors used in dark themes
$ct-colors-dark: (
```

```
  'grey': #697C93,
  'brown': #96663C,

  'yellow': #F8DC33,
  'gold': #CC9A0E,
  'orange': #F69F34,
  'red-orange': #FC5603,
  'red': #FF3948,
  'pink': #FF61F8,
  'purple': #D835FF,
  'violet': #AA59F9,
  'indigo': #7933FF,
  'blue': #69A1FF,
  'sky-blue': #33C7FF,
  'lemon-green': #ADD80F,
  'green-blue': #3DD5AF,
  'green': #4EDC7D,
) !default;
```

As you can tell, the defined color only mentions the *main* color. The *main* color refers to the **500** swatch of the color: `--{color}-500`, so in the case of `red` for example, it'll be `--red-500`. All other swatches are computed dynamically by `css-theming`.

If you want to override the color map in the theme, you can provide a similar map. You're free to define any number of colors from above, `css-theming` will fall back to the default for the other colors.

If you want to override the default for all themes, you can set `$ct-colors-light` or `$ct-colors-dark` to a new map of colors as long as they follow the same syntax. For example:

```
$ct-themes: (
  'default': (
    'brightness': 'light',
    'colors': (
      'red': #323232,
        ...
    )
  ),
  ...
);
```

Customizing semantic colors

Overriding semantic color targets is very similar, you'll have to provide a similar map to the following default:

```
$ct-semantic-colors: (
  'primary': 'purple',
  'success': 'green',
  'info': 'blue',
  'warning': 'orange',
  'danger': 'red',
) !default;
```

`$ct-semantic-colors` maps a *semantic color* to a *target color* from the color map in the same theme. For example: `primary` to `purple`, `success` to `green`, and so on.

The map assigns a semantic color name (i.e `primary`) to a color target name from the color map in the same theme (i.e `purple`).

Customizing swatches

This is the default definition of the swatches in `css-theming`:

```
$ct-swatches: (
  50: 15%,
  75: 30%,
  100: 40%,
  200: 50%,
  300: 60%,
  400: 80%,
  500: 100%,
) !default;
```

`css-theming` uses this map to compute color swatches. You're free to override this to anything you like. The **key** (i.e `50`, `75`, etc.) in the map will act as the *swatch's name*. For example, from this map, the following variables will be produced for each color:

```
--{color}-50: ...
--{color}-75: ...
--{color}-100: ...
```

```
...
```

The **value** in the map will be used as the percentage of the mixing between the color and the whitish/blackish color `css-theming` chooses, depending on the theme's brightness.

The same method follows for the rest of the props, you can always check the defaults in the src SCSS to figure out what you're supposed to override.

Adding More Theme Props

This lesson discusses how we can add custom theme props to our themes.

WE'LL COVER THE FOLLOWING

- Step 1: Add custom prop
- Step 2: Link to a CSS variable

Step 1: Add custom prop

Adding additional props to the theme definition is easy:

```
$ct-themes: (
  'default': (
    'brightness': 'light',

    'border-radius': 3px // We're adding our own defined theme prop here t
hat we'll use ourselves
  ),
  'default-dark': (
    'brightness': 'dark',

    'border-radius': 2px
  ),
);
```

In the above example, we added a `border-radius` prop to our themes. This allows us to change this `border-radius` value from theme to theme.

The next step is creating a dynamic CSS variable that points to this additional prop.

Step 2: Link to a CSS variable

> **Note:** A dynamic CSS variable in this context is one that changes its

> **Note**: A dynamic CSS variable in this context is one that changes its values depending on the active theme.

We do this by using a SCSS mixin defined in `css-theming`:

```
@include ct-themes-apply {
  --border-radius: #{map-get($theme, 'border-radius')};
}
```

> **Note**: A SCSS mixin simply defines some content that we can insert in multiple places.

The `ct-themes-apply` mixin allows you to define additional dynamic CSS variables tied with the theme. Inside, you'll have access to the resolved theme definition, in the `$theme` variable. So, all we have to do here is obtain the `border-radius` value from the theme using `map-get`:

```
map-get($theme, 'border-radius')
```

and create a new CSS variable from that:

```
--border-radius: #{SCSS EXPRESSION};
```

Now, we have a dynamic CSS variable that we can easily consume! This is the end result that we want to reach: a simple CSS variable that we can reference in our app.

Obtaining CSS Design Values

This lesson shows how to programmatically access design values.

WE'LL COVER THE FOLLOWING

- Obtaining defined themes

The `design-values` module in `css-theming` gives you programmatic access to some of the CSS defined variables. You can, for example, obtain and display all the semantic colors in your app:

```
import {
  getSemanticColorNames,
} from 'css-theming';

const semanticColorNames = getSemanticColorNames();
// Do whatever you want with `semanticColorNames`, it's an array of string
s.
```

> **Note:** All modules are exported in css-theming so you can import `from 'css-theming'` instead of `from 'design-values'`;

Here are all the definitions you can obtain from `design-values`:

```
swatchNames
fgSwatchNames
colorNames
semanticColorNames
themeCssNames
```

Obtaining defined themes

A common operation is displaying a list of your themes. For this we'll use the

`getThemes` function. This returns an array of `Theme` objects, defined as:

```
export interface Theme {
  category: string | null;
  name: string;
  cssName: string;
  brightness: Brightness;
}
```

So, `getThemes` might return something like:

```
[{
  "name": "default",
  "category": "",
  "cssName": "theme-default",
  "brightness": "light"
}, {
  "name": "default-dark",
  "category": "",
  "cssName": "theme-default-dark",
  "brightness": "dark"
}]
```

By getting design values in this way, we can easily create dynamic pages that show info about our theme, and more importantly, create a page where the user can select the theme. Next, we'll learn how to deal with multiple theme categories, something that a more complex app might need.

Registering Additional Theme Categories

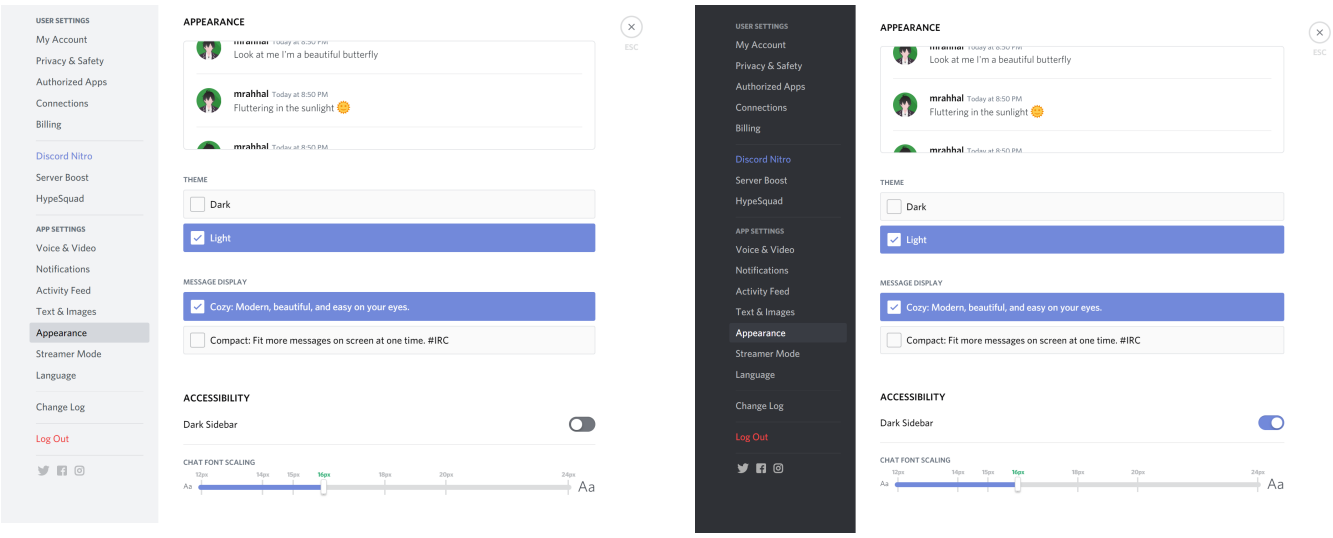
This lesson explains what is meant by registering an additional theme category and how to do it.

WE'LL COVER THE FOLLOWING

- Introduction
- Theme categories
  - Example

Introduction

Let's look at discord for a second:



You'll notice that you can change the sidebar's theme and that it becomes unrelated to the main theme of the app. That's really convenient, and this leads to the idea of having multiple **theme categories** as we'll call them.

Theme categories

Taking advantage of theme categories will allow you to create as many categories as you want so that you can apply themes at different levels of your app. Maybe you have some kind of a **launcher** page that you want to apply

app. Maybe you have some kind of a **launcher** page that you want to apply themes for. And maybe you also have some kind of *navbar* or *sidebar* to which

you also want to scope another category of themes. Most of the functions in `css-theming` takes an optional `category` parameter, so each category scopes its themes so that you can deal with them individually.

It's a very powerful feature of `css-theming` and we'll learn now how to use it.

Example

As an example, we'll be creating a theme category for the sidebar in our app.

First, we'll start by registering this new category. We use the `ct-themes-category-register` mixin for that:

```
// Don't forget this import in your variables files!
@import '../../node_modules/css-theming/src/scss/pure';

// Register a new themes category that we'll use for the sidebar
$ct-sidebar-themes: (
  'default': (
    'brightness': 'light',
  ),
  'default-dark': (
    'brightness': 'dark',
  )
);

@include ct-themes-category-register('sidebar', $ct-sidebar-themes, $applied-at: '.sidebar');
```

> **Note:** The default/main theme category of the app is `''` (empty string) in CSS, and `null` in javascript. The `Theme` object contains this `category` property.

As you can see, we just created a new theme category called **sidebar** with 2 themes inside of it. We have to provide that name when calling the mixin, the themes map, and also where you want to apply the theme.

This `.sidebar` is the element that the theme's CSS variables will be set in.

Next, we have to make sure to call `initializeTheming` for this category.

Next, we have to make sure to call `initializeTheming` for this category:

```
// Initializes the main default category
initializeTheming();

// Initializes the sidebar category
initializeTheming( /* theme */ null, /* category */ 'sidebar');
```

Now you are familiar with how to use `css-theming` in your app. The next section shows some samples to see how `css-theming` works in practice and across many use cases.

About the Samples

This lesson provides samples for visualizing what you've studied so far in the course.

WE'LL COVER THE FOLLOWING

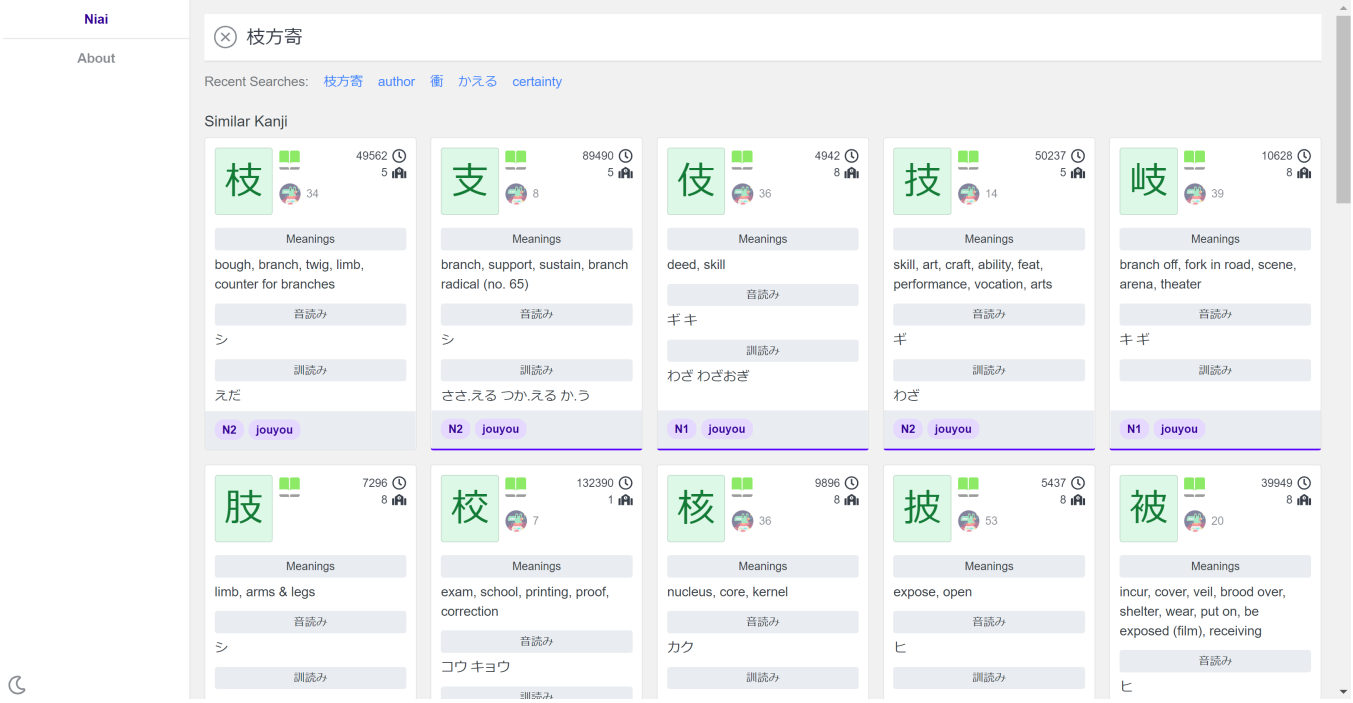
- Niai
- Basic
- Additional theme props
- Additional sidebar theme category
- Additional navbar theme category

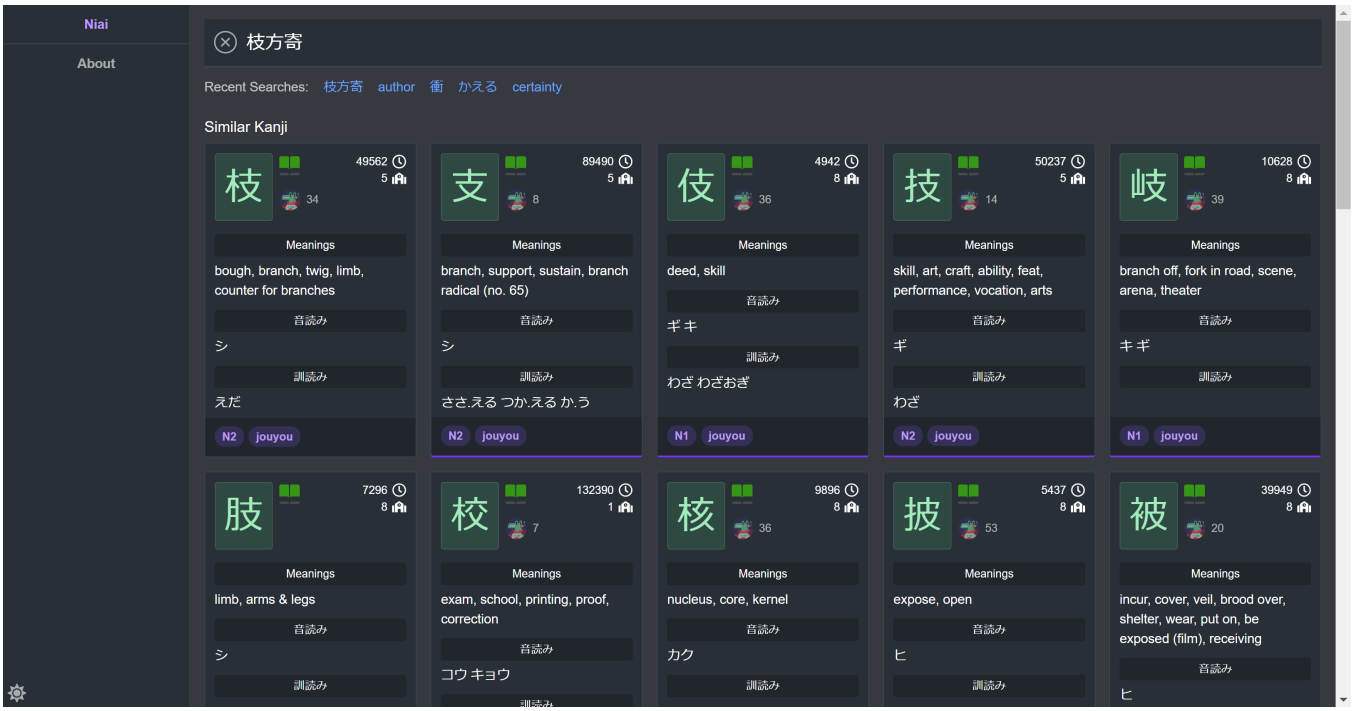
All samples are included in the repository under the samples folder. You can run any sample by executing the following commands inside a certain sample's folder:

```
$ npm i
$ npm start
```

Niai

Niai is a modern app for looking up similar Kanjis, homonyms, and synonyms in the Japanese language, which is also open sourced here! So, you can check it for a real-world usage of `css-theming`.





Basic

The basic sample demonstrates how to setup `css-theming` in an app. We don't do anything special here, we only initialize the default theme category.

Additional theme props

The additional-theme-props sample demonstrates how to add additional theme props to be consumed in the app.

Additional sidebar theme category

The additional-sidebar-theme-category sample demonstrates how to register a sidebar theme category.

Additional navbar theme category

The additional-navbar-theme-category sample demonstrates how to register a sidebar theme category.

Do you have any suggestions for a new sample? Be sure to reach out to me about it!